

数据结构

第五章 数组和广义表

数组的定义

数组的顺序存储结构

矩阵的压缩存储

广义表的定义

广义表的存储结构

第一节数组的定义

数组可以看成是一种特殊的线性表，即线性表中数据元素本身也是一个线性表

■ 定义

$$A_{m \times n} = \begin{bmatrix} \overbrace{(a_{11} \ a_{12} \ \dots \ a_{1n})} \\ \overbrace{(a_{21} \ a_{22} \ \dots \ a_{2n})} \\ \overbrace{(\dots \ \dots \ \dots \ \dots)} \\ \overbrace{(a_{m1} \ a_{m2} \ \dots \ a_{mn})} \end{bmatrix}$$

■ 数组特点

- ◇ 数组结构固定
- ◇ 数据元素同构

■ 数组运算

- ◇ 给定一组下标，存取相应的数据元素
- ◇ 给定一组下标，修改数据元素的值

第二节数组的顺序存储结构

□ 次序约定

■ 以行序为主序

■ 以列序为主序

a_{11}	a_{12}		a_{1n}
a_{21}	a_{22}		a_{2n}
.....			
a_{m1}	a_{m2}		a_{mn}

a_{11}	a_{11}
a_{12}	a_{21}
...	...
a_{1n}	a_{m1}
a_{21}	a_{12}
a_{22}	a_{22}
...	...
a_{2n}	a_{m2}
...	...
a_{n1}	a_{1n}
a_{m2}	a_{2n}
...	...
a_{nn}	a_{mn}

列序: $Loc(a_{ij}) = Loc(a_{11}) + [(j-1)m + (i-1)] * l$

第三节 矩阵的压缩存储

- ☐ 对称矩阵
- ☐ 三角矩阵
- ☐ 对角矩阵
- ☐ 稀疏矩阵

稀疏矩阵的压缩存储方法

对称矩阵

$$\begin{bmatrix}
 \mathbf{a_{11}} & \mathbf{a_{12}} & \cdots & \cdots & \cdots & \mathbf{a_{1n}} \\
 \mathbf{a_{21}} & \mathbf{a_{22}} & \cdots & \cdots & \cdots & \mathbf{a_{2n}} \\
 & & \cdots & \cdots & \cdots & \\
 \mathbf{a_{n1}} & \mathbf{a_{n2}} & \cdots & \cdots & \cdots & \mathbf{a_{nn}}
 \end{bmatrix}$$

按行序为主序：

a11	a21	a22	a31	a32		an1		ann
k=0	1	2	3	4		n(n-1)/2		n(n+1)/2-1

$$k = \begin{cases} i(i-1)/2 + j - 1, & i \geq j \\ j(j-1)/2 + i - 1, & i < j \end{cases}$$

三角矩阵

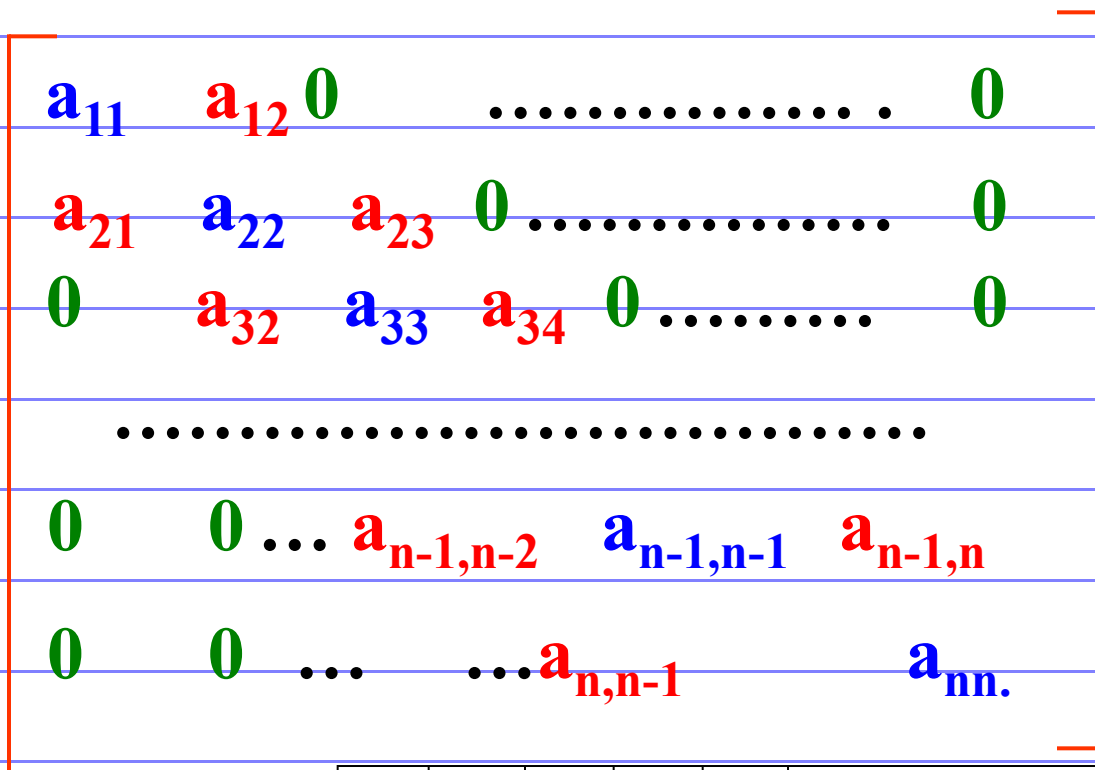
$$\begin{bmatrix}
 a_{11} & 0 & 0 & \dots & 0 \\
 a_{21} & a_{22} & 0 & \dots & 0 \\
 \dots & \dots & \dots & \dots & 0 \\
 a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn}
 \end{bmatrix}$$

按行序为主序：

a ₁₁	a ₂₁	a ₂₂	a ₃₁	a ₃₂		a _{n1}		a _{nn}
k=0	1	2	3	4		n(n-1)/2		n(n+1)/2-1

$$\text{Loc}(a_{ij}) = \text{Loc}(a_{11}) + [i(i-1)/2 + (j-1)] * l$$

对角矩阵



按行序为主序：

a11	a12	a21	a22	a23			an(n-1)	ann
k=0	1	2	3	4		n(n-1)/2		n(n+1)/2-1

$$\text{Loc}(a_{ij}) = \text{Loc}(a_{11}) + 2(i-1) + (j-1)$$

稀疏矩阵

- ✧定义：非零元较零元少，且分布没有一定规律的矩阵
- ✧压缩存储原则：只存矩阵的行列维数和每个非零元的行列下标及其值

$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

M由{(1,2,12), (1,3,9), (3,1,-3), (3,6,14), (4,3,24),
(5,2,18), (6,1,15), (6,4,-7)} 和矩阵维数 (6,7) 唯一确定

◇稀疏矩阵的压缩存储方法

◇三元组表

$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

行列下标

非零元值

	i	j	v
0	6	7	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	6	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

ma[0].i,ma[0].j,ma[0].v分别存放矩阵
行列维数和非零元个数

三元组表所需存储单元个数为 $3(t+1)$
其中t为非零元个数

8

ma

三元组顺序表

```
#define MAXSIZE 12500
```

```
typedef struct {
```

```
    int i, j;
```

```
    ElemType e;
```

```
} Triple;
```

```
typedef struct {
```

```
    Triple data[MAXSIZE+1];
```

```
    int mu, nu, tu;
```

```
}
```

带辅助行向量的二元组表

增加一个辅助数组NRA[m+1]，其物理意义是第i行第一个非零元在二元组表中的起始地址（m为行数）

显然有：
$$\begin{cases} \text{NRA}[1]=1 \\ \text{NRA}[i]=\text{NRA}[i-1]+\text{第}i-1\text{行非零元个数}(i\geq 2) \end{cases}$$

列下标和非零元值

NRA[0]不用或存矩阵行数

	NRA	j	v
0	6	7	8
1	1	2	12
2	3	3	9
3	3	1	-3
4	5	6	14
5	6	3	24
6	7	2	18
		1	15
		4	-7
		ma	

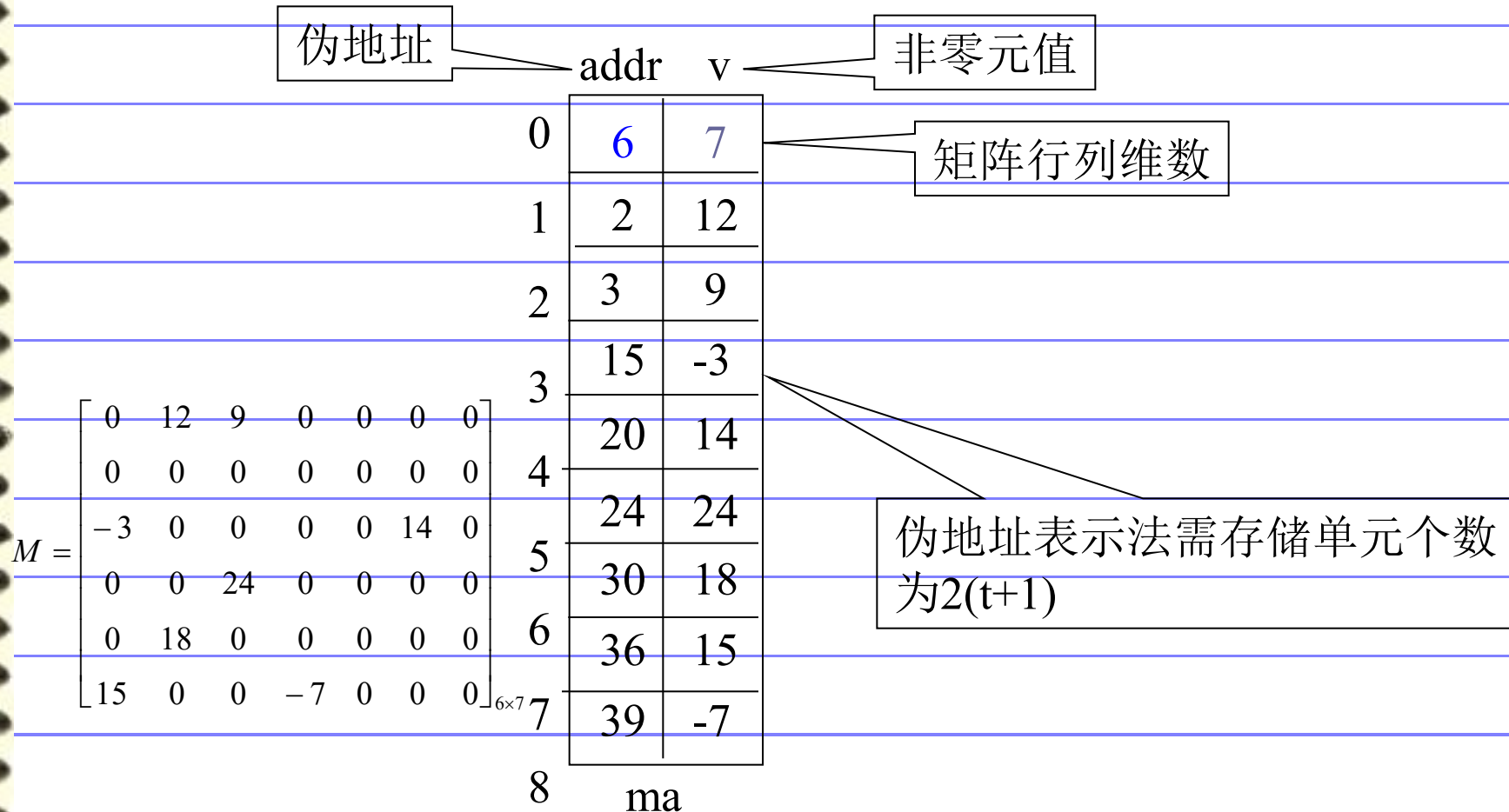
矩阵列数和非零元个数

二元组表需存储单元个数为 $2(t+1)+m+1$

$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

伪地址表示法

伪地址：本元素在矩阵中（包括零元素在内）
按行优先顺序的相对位置



求转置矩阵

☆ 问题描述：已知一个稀疏矩阵的三元组表，求该矩阵转置矩阵的三元组表

☆ 问题分析

一般矩阵转置算法：

```
for(col=0;col<n;col++)  
    for(row=0;row<m;row++)  
        n[col][row]=m[row][col];  
T(n)=O(m×n)
```

$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

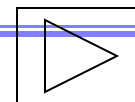
$$N = \begin{bmatrix} 0 & 0 & -3 & 0 & 0 & 15 \\ 12 & 0 & 0 & 0 & 18 & 0 \\ 9 & 0 & 0 & 24 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -7 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 14 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}_{7 \times 6}$$

ma

	i	j	v
0	6	7	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	6	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

	i	j	v
0	7	6	8
1	1	3	-3
2	1	6	15
3	2	1	12
4	2	5	18
5	3	1	9
6	3	4	24
7	4	6	-7
8	6	3	14

mb



☆解决思路：只要做到

①将矩阵行、列维数互换

②将每个三元组中的 i 和 j 相互调换

③重排三元组次序，使 mb 中元素以 N 的行(M 的列)为主序

方法一：按 M 的列序转置

即按 mb 中三元组次序依次在 ma 中找到相应的三元组进行转置。

为找到 M 中每一列所有非零元素，需对其三元组表 ma 从第一行起扫描一遍。由于 ma 中以 M 行序为主序，所以由此得到的恰是 mb 中应有的顺序。

```
Status TransposeSMatrix(TSMatrix M, TSMatrix &T) {
```

```
    T.mu = M.nu; T.nu = M.mu; T.tu = M.tu;
```

```
    if(T.tu) {
```

```
        q=1;
```

```
        for(col=1; col<=M.nu; ++col)
```

```
            for(p=1; p<=M.tu; ++p)
```

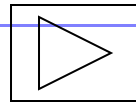
```
                if(M.data[p].j == col) {
```

```
                    T.data[q].i = M.data[p].j; T.data[q].j = M.data[p].i;
```

```
                    T.data[q].e = M.data[p].e; ++q; }
```

```
    }
```

```
    return OK;
```



☆ 算法描述:

☆ 算法分析: $T(n) = O(M \text{ 的列数 } n \times \text{非零元个数 } t)$

☆ 若 t 与 $m \times n$ 同数量级, 则 $T(n) = O(m \times n^2)$

	i	j	v	
0	6	7	8	
p→1	1	2	12	←p
p→2	1	3	9	←p
p→3	3	1	-3	←p
p→4	3	6	14	←p
p→5	4	3	24	←p
p→6	5	2	18	←p
p→7	6	1	15	←p
p→8	6	4	-7	←p
	ma			

col=1

	i	j	v	
0	7	6	8	
k→1	1	3	-3	
k→2	1	6	15	
k→3	2	1	12	
k→4	2	5	18	
k→5	3	1	9	
6	3	4	24	
7	4	6	-7	
8	6	3	14	
	mb			

col=2

方法二：快速转置

即按ma中三元组次序转置，转置结果放入mb中恰当位置。

此法关键是要预先确定M中每一列第一个非零元在mb中位置，为确定这些位置，转置前应先求得M的每一列中非零元个数

实现：设两个数组

num[col]：表示矩阵M中第col列中非零元个数

cpot[col]：指示M中第col列第一个非零元在mb中位置

显然有：
$$\begin{cases} \text{cpot}[1]=1; \\ \text{cpot}[\text{col}]=\text{cpot}[\text{col}-1]+\text{num}[\text{col}-1]; & (2 \leq \text{col} \leq \text{ma}[0].j) \end{cases}$$

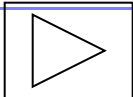
$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
cpot[col]	1	3	5	7	8	8	9

```

Status FastTransposeSMatrix(TSMatrix M, TSMatrix &T) {
    T.mu = M.nu; T.nu = M.mu; T.tu = M.tu;
    if(T.tu) {
        for(col=1; col<=M.nu; ++col) num[col]=0;
        for(t=1; t<=M.tu, ++t) ++num[M.data[t].j];
        cpot[1] = 1;
        for(col=2; col<=M.nu; ++col) cpot[col] = cpot[col-1] + num[col-1];
        for(p=1; p<=M.tu; ++p) {
            col = M.data[p].j  q = cpot[col];
            T.data[q].i = M.data[p].j; T.data[q].j = M.data[p].i;
            T.data[q].e = M.data[p].e; ++cpot[col]; }
        }
    return OK;
}

```

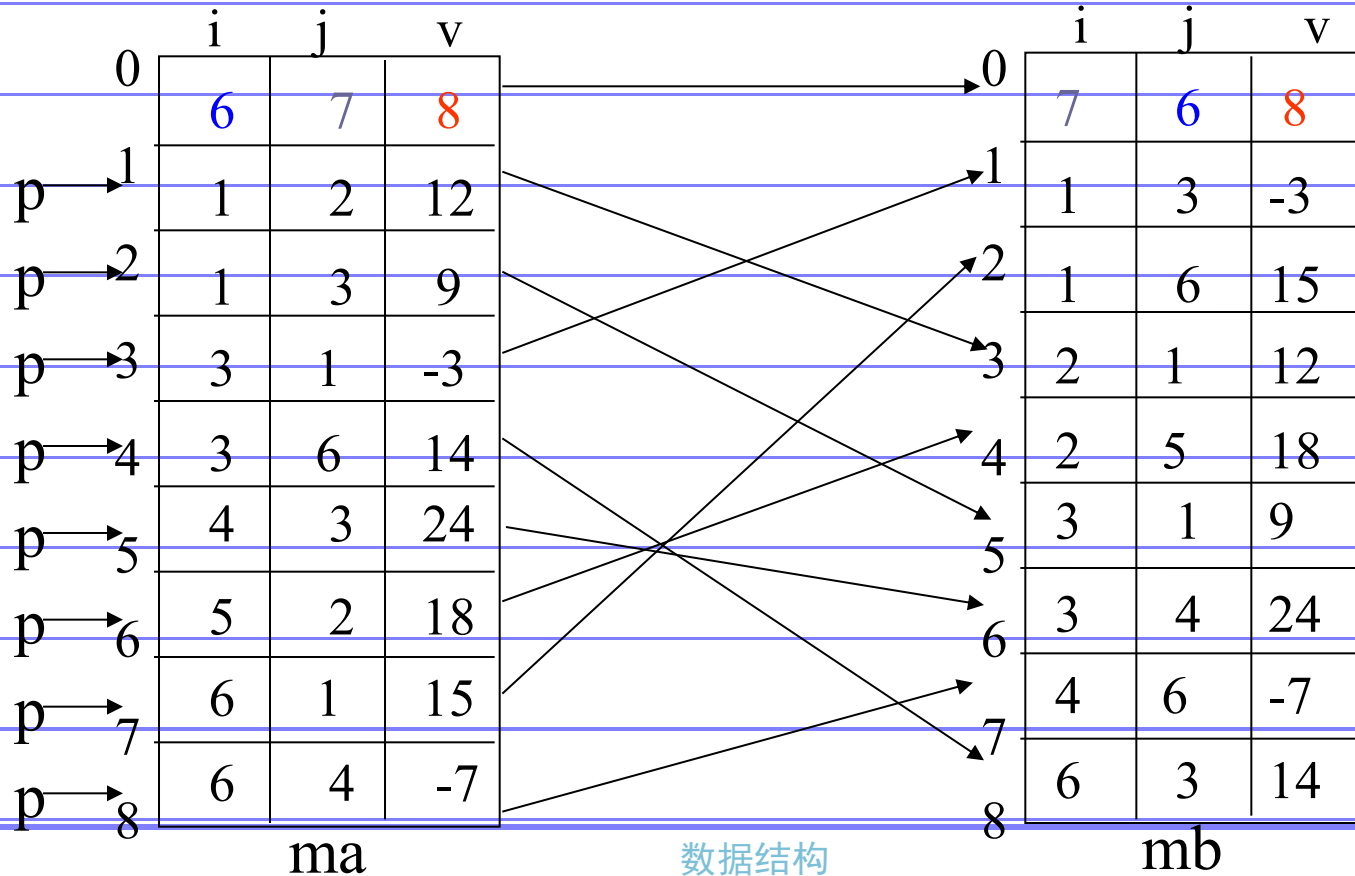
☆算法描述： 

☆算法分析： $T(n) = O(M \text{的列数} n + \text{非零元个数} t)$
若 t 与 $m \times n$ 同数量级，则 $T(n) = O(m \times n)$

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
cpot[col]	1	3	5	7	8	8	9

2 4 6 9

3 5 7



链式存储结构

□ 带行指针向量的单链表表示

☆ 每行的非零元用一个单链表存放

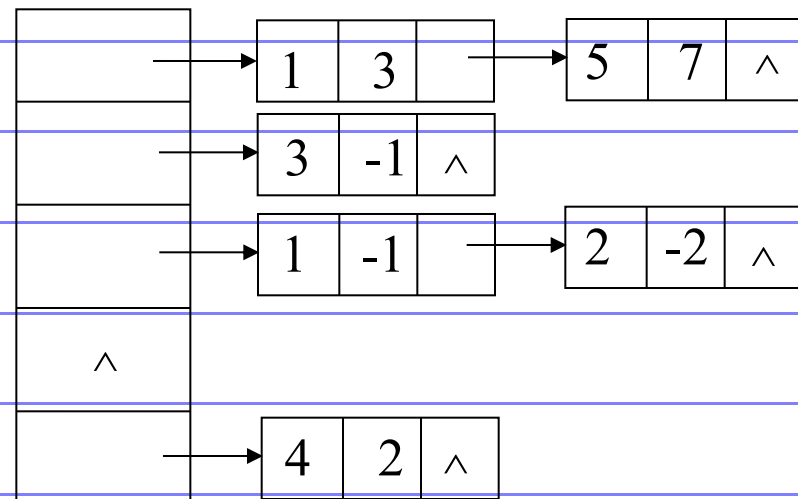
☆ 设置一个行指针数组，指向本行第一个非零元结点；若本行无非零元，则指针为空

```
typedef struct node
```

```
{  int col;
    int val;
    struct node *link;
}JD;
```

```
typedef struct node *TD;
```

$$A = \begin{bmatrix} 3 & 0 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 & 0 \\ -1 & -2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \end{bmatrix}$$



数据结构

需存储单元个数为 $3t+m$

十字链表

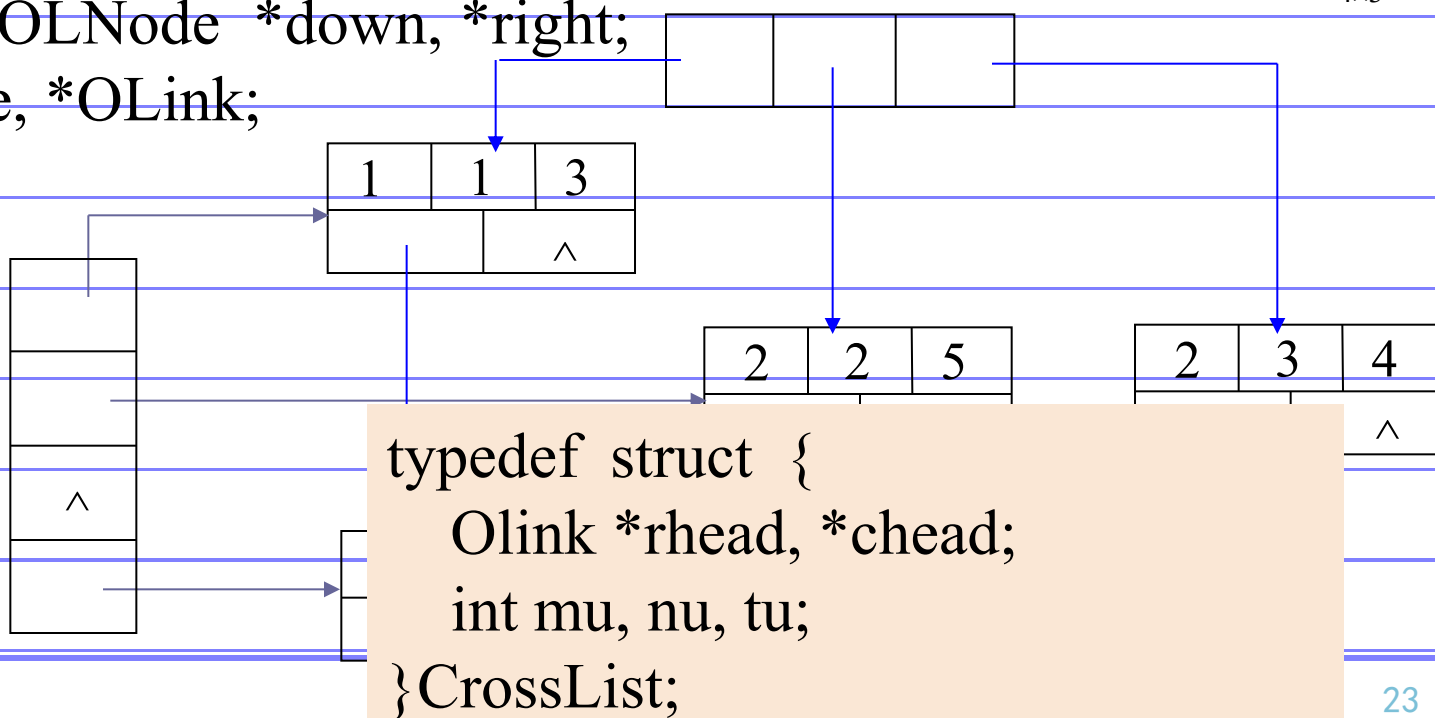
☆ 设行指针数组和列指针数组，分别指向每行、列第一个非零元

☆ 结点定义

```
typedef struct OLNode {
    int i, j;
    ElemType e;
    struct OLNode *down, *right;
} OLNode, *OLink;
```

row	col	val
	down	right

$$A = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 5 & 4 \\ 0 & 0 & 0 \\ 8 & 0 & 0 \end{bmatrix}_{4 \times 3}$$



第四节 广义表的定义

广义表是线性表的推广和扩充，在人工智能领域中应用十分广泛。

在第2章中，我们把线性表定义为 $n(n \geq 0)$ 个元素 $a_1, a_2, \dots, a_n$ 的有穷序列，该序列中的所有元素具有相同的数据类型且只能是原子项(Atom)。所谓原子项可以是一个数或一个结构，是指结构上不可再分的。若放松对元素的这种限制，容许它们具有其自身结构，就产生了广义表的概念。

广义表(Lists，又称为列表)：是由 $n(n \geq 0)$ 个元素组成的有穷序列： $LS = (a_1, a_2, \dots, a_n)$

其中 a_i 或者是原子项，或者是一个广义表。LS是广义表的名字， n 为它的长度。若 a_i 是广义表，则称为LS的子表。

习惯上：原子用小写字母，子表用大写字母。

若广义表LS非空时：

- ◆ a_1 (表中第一个元素)称为表头；
- ◆ 其余元素组成的子表称为表尾；($a_2, a_3, \dots, a_n$)
- ◆ 广义表中所包含的元素(包括原子和子表)的个数称为表的长度。
- ◆ 广义表中括号的最大层数称为表深(度)。

表5-2 广义表及其示例

有关广义表的这些概念的例子如下表所示。

广 义 表	表长n	表深h
$A=()$	0	0
$B=(e)$	1	1
$C=(a,(b,c,d))$	2	2
$D=(A,B,C)$	3	3
$E=(a,E)$	2	∞
$F=(())$	1	1

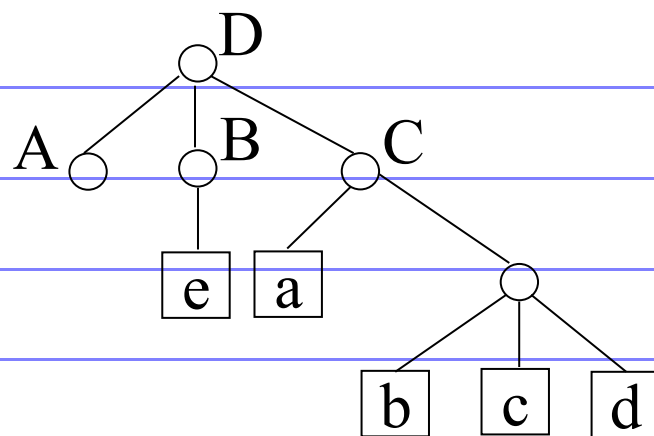


图 广义表的图形表示

广义表的重要结论：

- (1) 广义表的元素可以是原子，也可以是子表，子表的元素又可以是子表，...。即广义表是一个多层次的结构。
- (2) 广义表可以被其它广义表所共享，也可以共享其它广义表。广义表共享其它广义表时通过表名引用。
- (3) 广义表本身可以是一个递归表，即列表也可以是其本身的一个子表。
- (4) 根据对表头、表尾的定义，任何一个非空广义表的表头可以是原子，也可以是子表，而表尾必定是广义表。
- (5) 值得提醒的是列表 $()$ 和 $(())$ 不同。前者为空表，长度 $n=0$ ；后者长度 $n=1$ ，可分解得到其表头、表尾均为空表 $()$

广义表的存储结构

由于广义表中的数据元素具有不同的结构，通常用链式存储结构表示，每个数据元素用一个结点表示。因此，广义表中就有两类结点：

- ◆ 一类是表结点，用来表示广义表项，由标志域，表头指针域，表尾指针域组成；
- ◆ 另一类是原子结点，用来表示原子项，由标志域，原子的值域组成。如图5-13所示。

只要广义表非空，都是由表头和表尾组成。即一个确定的表头和表尾就唯一确定一个广义表。

标志tag=0	原子的值
---------	------

(a) 原子结点

标志tag=1	表头指针hp	表尾指针tp
---------	--------	--------

(b) 表结点

图 广义表的链表结点结构示意图

相应的数据结构定义如下：

```
typedef struct GLNode
```

```
{ int tag; /* 标志域，为1：表结点；为0：原子结点 */
```

```
union
```

```
{ elemtype value; /* 原子结点的值域 */
```

```
struct
```

```
{ struct GLNode *hp, *tp;
```

```
}ptr; /* ptr和atom两成员共用 */
```

```
}Gdata;
```

```
} GLNode; /* 广义表结点类型 */
```

例：对 $A=()$ ， $B=(e)$ ， $C=(a, (b, c, d))$ ， $D=(A, B, C)$ ， $E=(a, E)$ 的广义表的存储结构如图5-14所示。

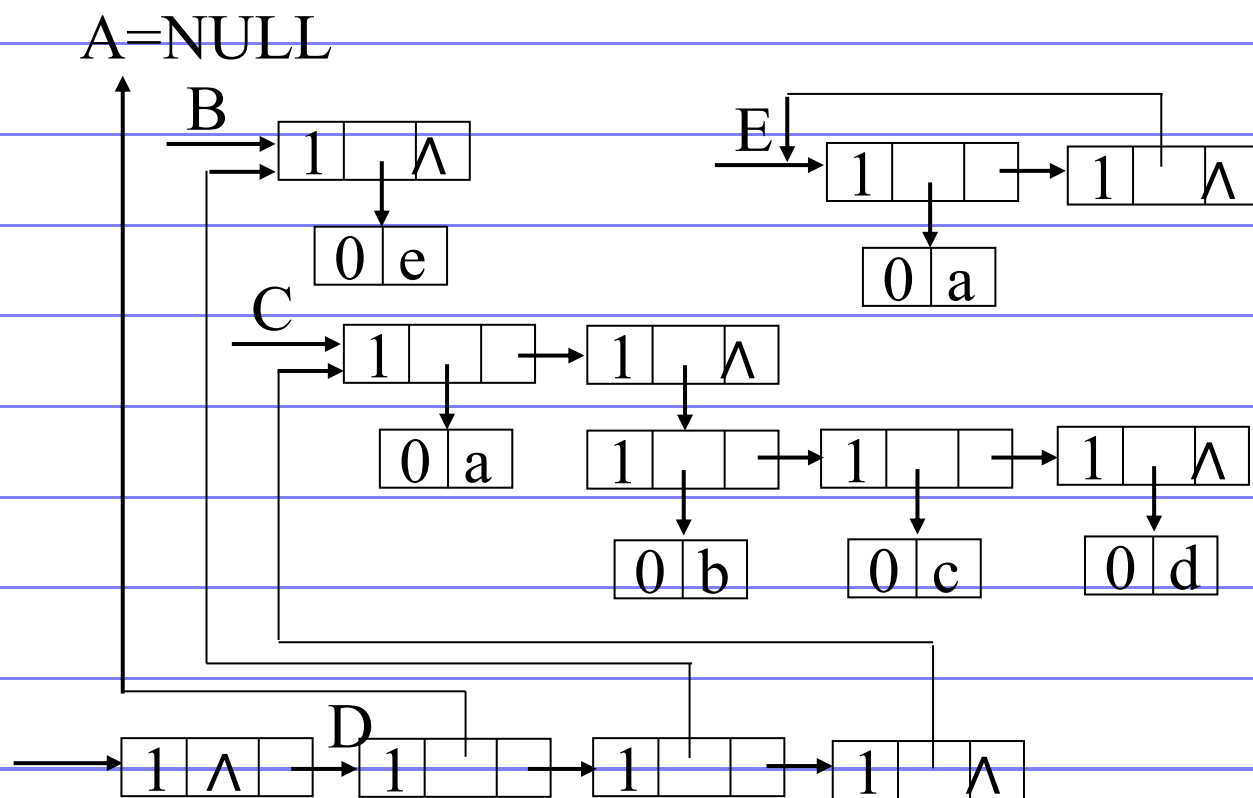


图5-14 广义表的存储结构示意图

对于上述存储结构，有如下几个特点：

- (1) 若广义表为空，表头指针为空；否则，表头指针总是指向一个表结点，其中hp指向广义表的表头结点(或为原子结点，或为表结点)，tp指向广义表的表尾(表尾为空时，指针为空，否则必为表结点)。
- (2) 这种结构求广义表的长度、深度、表头、表尾的操作十分方便。

习题五

(1) 什么是广义表？请简述广义表与线性表的区别？

(2) 一个广义表是 $(a, (a, b), d, e, (a, (i, j), k))$ ，请画出该广义表的链式存储结构。

(3) 设有二维数组 $a[6][8]$ ，每个元素占相邻的4个字节，存储器按字节编址，已知 a 的起始地址是1000，试计算：

① 数组 a 的最后一个元素 $a[5][7]$ 起始地址；

② 按行序优先时，元素 $a[4][6]$ 起始地址；

③ 按行序优先时，元素 $a[4][6]$ 起始地址。

数据结构题集（C语言版）：5.9，5.18，5.21

(4) 设A和B是稀疏矩阵，都以三元组作为存储结构，请写出矩阵相加的算法，其结果存放在三元组表C中，并分析时间复杂度。

(5) 设有稀疏矩阵B如下图所示，请画出该稀疏矩阵的三元组表和十字链表存储结构。

$$A = \begin{pmatrix} 0 & 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 0 & 0 & 4 \\ 0 & 0 & 2 & 0 & 0 & 2 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 4 & 0 & 5 & 0 \\ 0 & 0 & -3 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$